

Detección Automática de Degradación Operacional en Kubernetes

Clasificación y explicabilidad para observabilidad en Kubernetes

Darieth Farid Sánchez Velásquez

dfsanchezv@eafit.edu.co

Valentina Benitez Zapata

vbenitez@eafit.edu.co

Escuela de Ciencias Aplicadas e Ingeniería . Universidad EAFIT

Curso: Inteligencia Artificial . Semestre 2026-1

Repositorio: https://github.com/DariethSV/final_project_IA/tree/main

Resumen

La observabilidad en Kubernetes representa un desafío debido a la complejidad de los entornos cloud-native y la necesidad de detectar anomalías de manera temprana. En este contexto, las técnicas de Inteligencia Artificial permiten automatizar procesos de monitoreo y diagnóstico operacional mediante el análisis de métricas y eventos del sistema.

El objetivo de este proyecto es desarrollar un sistema capaz de detectar automáticamente estados degradados en clústeres Kubernetes mediante técnicas de Machine Learning y generar explicaciones operacionales utilizando LLM.

La metodología propuesta incluye preprocesamiento, análisis exploratorio de datos (EDA), reducción de dimensionalidad con PCA y entrenamiento de modelos de clasificación como Random Forest y XGBoost. Adicionalmente, se utiliza SHAP para explicabilidad y un componente LLM para generar interpretaciones en lenguaje natural orientadas a observabilidad.

Se espera obtener métricas de desempeño superiores al baseline tradicional utilizando indicadores como Accuracy, Precision, Recall, F1-score y ROC-AUC. Además, el componente LLM busca facilitar la interpretación de las predicciones y apoyar el análisis operacional del clúster.

Como contribución principal, el proyecto propone un enfoque de observabilidad inteligente para Kubernetes basado en clasificación automática y explicabilidad operacional. Como trabajo futuro, se plantea integrar datos en tiempo real mediante Prometheus e incorporar arquitecturas RAG y agentes autónomos para automatizar recomendaciones de mitigación.

Palabras clave: Machine Learning, SHAP, LLM, Kubernetes, Observabilidad.

1. Introducción

La adopción de Kubernetes como plataforma de orquestación de contenedores ha incrementado significativamente en los últimos años debido a su capacidad para desplegar y administrar aplicaciones distribuidas en entornos cloud-native. Sin embargo, la complejidad operacional de estos sistemas genera desafíos relacionados con monitoreo, observabilidad y detección temprana de anomalías. Problemas como sobrecarga de recursos, tráfico anómalo y degradación de servicios pueden afectar la disponibilidad y el rendimiento del clúster, haciendo necesario el uso de mecanismos automáticos de detección y análisis.

En este contexto, las técnicas de Inteligencia Artificial y Machine Learning han demostrado ser herramientas efectivas para identificar patrones anómalos a partir de métricas y eventos del sistema. Adicionalmente, el uso de modelos LLM permite complementar estos sistemas mediante la generación de explicaciones en lenguaje natural, facilitando la interpretabilidad de las predicciones y el diagnóstico operacional.

El presente proyecto propone un sistema para la detección automática de degradación operacional en Kubernetes utilizando técnicas de clasificación multiclase. Para ello, se emplean etapas de preprocesamiento, análisis exploratorio de datos (EDA), reducción de dimensionalidad mediante PCA, modelos de clasificación basados en XGBoost y mecanismos de explicabilidad utilizando SHAP. Finalmente, un modelo LLM genera interpretaciones operacionales en lenguaje natural a partir de las predicciones del sistema.

Aunque el dataset seleccionado fue diseñado originalmente para intrusion detection, las etiquetas presentes representan comportamientos anómalos que pueden reinterpretarse como estados degradados del clúster Kubernetes desde una perspectiva de observabilidad operacional. Para el desarrollo del proyecto se utiliza el dataset público “Kubernetes Intrusion Detection Datasets”

disponible en Kaggle, el cual contiene información relacionada con actividad normal y patrones anómalos asociados a ataques DoS en entornos Kubernetes. El problema se aborda como una tarea de clasificación multiclase con tres etiquetas: Normal Activity, DoS Attack 1 y DoS Attack 2.

La pregunta de investigación que guía este trabajo es la siguiente:

¿Puede un modelo de Machine Learning identificar correctamente estados operacionales en Kubernetes con una Accuracy superior a un baseline tradicional?

La metodología contempla etapas de entrenamiento y evaluación de modelos de Machine Learning utilizando métricas como Accuracy, Precision, Recall, F1-score y ROC-AUC, además de análisis de explicabilidad mediante SHAP y generación de explicaciones operacionales usando modelos LLM. Finalmente, el documento se organiza en secciones que describen el dataset y el EDA, la arquitectura del sistema, la metodología experimental, los resultados obtenidos, la discusión de hallazgos y las conclusiones del proyecto.

2. Datos y Exploración (EDA)

2.1. Descripción del dataset

Cuadro 1: Resumen del dataset utilizado

Atributo	Descripción
Fuente	https://www.kaggle.com/datasets/redamorsli/kube-ids0?resource=download
Registros (N)	10069
Features (p)	225 numéricas
Variable objetivo	label (multiclase)
Período / Versión	Versión 1 (2025)
Tamaño	14.030KB

2.2. Hallazgos del análisis exploratorio

El análisis exploratorio evidenció un desbalance significativo en la variable objetivo (label), donde la clase correspondiente al tráfico normal predominó ampliamente sobre las clases asociadas a ataques. Este comportamiento representa un reto importante para el entrenamiento de los modelos, ya que puede generar sesgos hacia la clase mayoritaria.

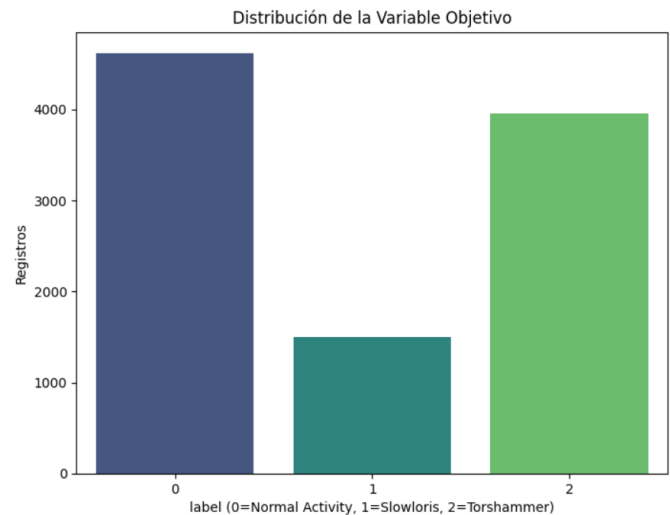


Figura 1: Distribución de la Variable Objetivo (label). 0=Normal Activity, 1=DoS Attack 1, 2=DoS Attack 2

En la distribución de las características seleccionadas, los boxplots permitieron identificar patrones relevantes:

- Torshammer (label 2) presentó distribuciones altamente uniformes con respecto a las tres características, algo que no ocurre en Normal Activity ni en Slowloris.
- Slowloris (label 1) no tiene un patrón claro para estas características, o los pocos registros no permiten observar su comportamiento.
- Normal Activity (label 0) es el único que presenta alta variabilidad y valores atípicos en ambas direcciones, reflejando la dinámica real de un clúster Kubernetes.
- La variable `container_memory_usage_bytes` presenta un comportamiento muy similar entre las tres clases del label, diferenciándose únicamente por algunos valores atípicos asociados a Normal Activity (label 0). Debido a esta baja capacidad de separación entre clases, esta característica no resulta especialmente discriminativa para el proceso de clasificación.

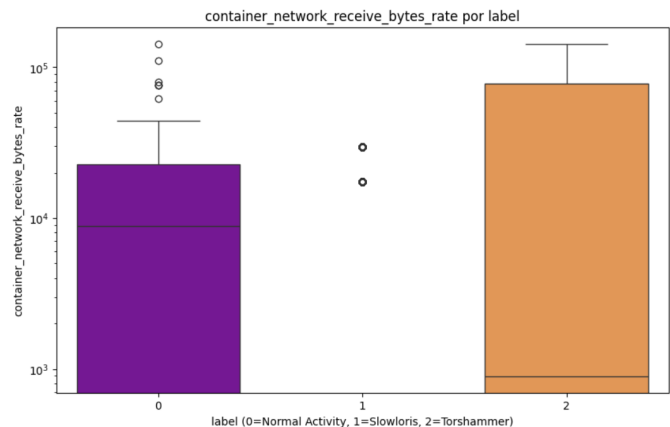


Figura 2: Distribución de `container_network_receive_bytes_rate` por label. 0=Normal Activity, 1=DoS Attack 1, 2=DoS Attack 2

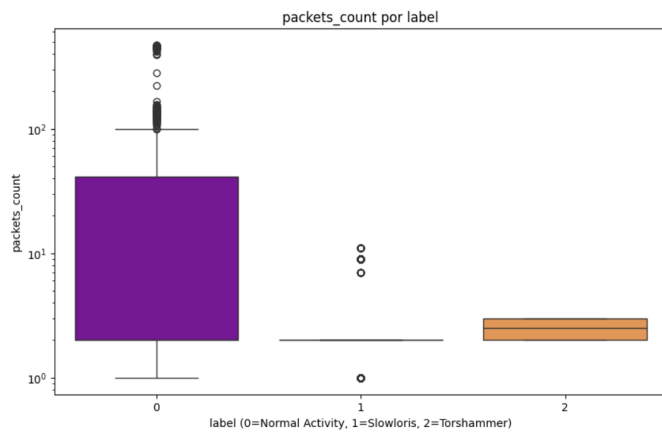


Figura 3: Distribución de packets_count por label. 0=Normal Activity, 1=DoS Attack 1, 2=DoS Attack 2

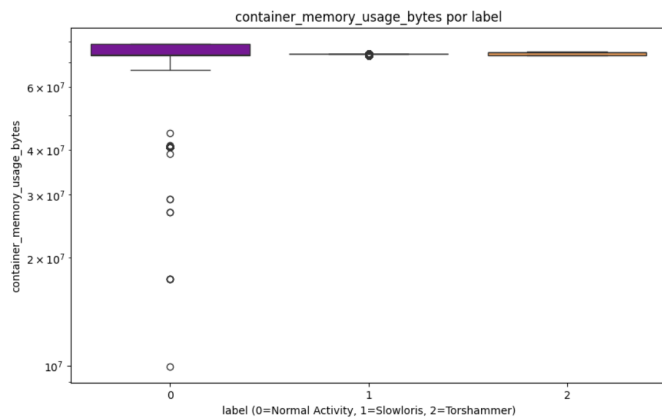


Figura 4: Distribución de container_memory_usage_bytes por label. 0=Normal Activity, 1=DoS Attack 1, 2=DoS Attack 2

El análisis de valores nulos indicó que el conjunto de datos no contenía registros faltantes, por lo que no fue necesario aplicar estrategias de imputación.

Las variables con mayor correlación positiva con label (0.688) son métricas de intervalos entre paquetes en dirección inversa, lo que indica que a medida que aumenta la severidad del ataque (Normal → Slowloris → Torshammer), los tiempos entre paquetes de respuesta se vuelven más regulares y predecibles. Por otro lado, las correlaciones negativas más fuertes (-0.57 a -0.53) corresponden a tamaños de segmento y flags PSH, sugiriendo que los ataques reducen drásticamente la variabilidad en el tamaño de los paquetes y el uso de flags de empuje, típicos del tráfico normal interactivo. En conjunto, esto refleja que los ataques DoS generan tráfico artificialmente uniforme y sin contenido útil, mientras que el tráfico normal de Kubernetes es más irregular y rico en variabilidad.

La siguiente tabla presenta los valores promedio de las características (C1=total_payload_bytes,

C2=container_network_receive_packets_rate, C3=container_sockets) que muestran mayor discrepancia entre las tres clases:

Cuadro 2: Características con mayor discrepancia entre clases

Clase	C1	C2	C3
Normal Activity (0)	52121	65	436
Slowloris (1)	44	21	138
Torshammer (2)	0	510	515

Estas diferencias tan marcadas reflejan la naturaleza intrínseca de cada comportamiento. En un clúster normal, las aplicaciones intercambian datos reales (payload alto), generan tráfico de red moderado y variable, y mantienen un número significativo de sockets por la operación legítima de múltiples servicios. Slowloris envía cabeceras HTTP incompletas de forma muy lenta y con payload mínimo (44 bytes por conexión), no recibe respuesta significativa, abre pocos sockets para no saturarse localmente y su tasa de paquetes es baja. Torshammer, en cambio, utiliza conexiones HTTP completas pero repetitivas sin payload, genera una tasa altísima de paquetes por segundo (510) para abrumar el objetivo, y abre muchos sockets (515) para mantener múltiples conexiones paralelas. Así, los valores extremos de Torshammer revelan su naturaleza agresiva y volumétrica, mientras que los valores intermedios pero muy bajos de Slowloris evidencian su estrategia de "bajo y lento" para evadir detección.

3. Arquitectura del Sistema

El pipeline tiene tres grandes secciones:

Preparación de datos: El dataset pasa por preprocesamiento y EDA, y luego por PCA para reducir dimensionalidad antes de entrar al modelo.

Motor de predicción: XGBoost recibe las componentes principales y genera la predicción. SHAP actúa como capa de interpretabilidad sobre esa predicción, identificando cuáles variables contribuyeron más al resultado.

Explicación en lenguaje natural: Tanto la predicción como las variables SHAP se empaquetan juntas en un prompt estructurado, que es lo que recibe el LLM. Esto le da al modelo de lenguaje el contexto exacto para generar una explicación operacional útil para el negocio.

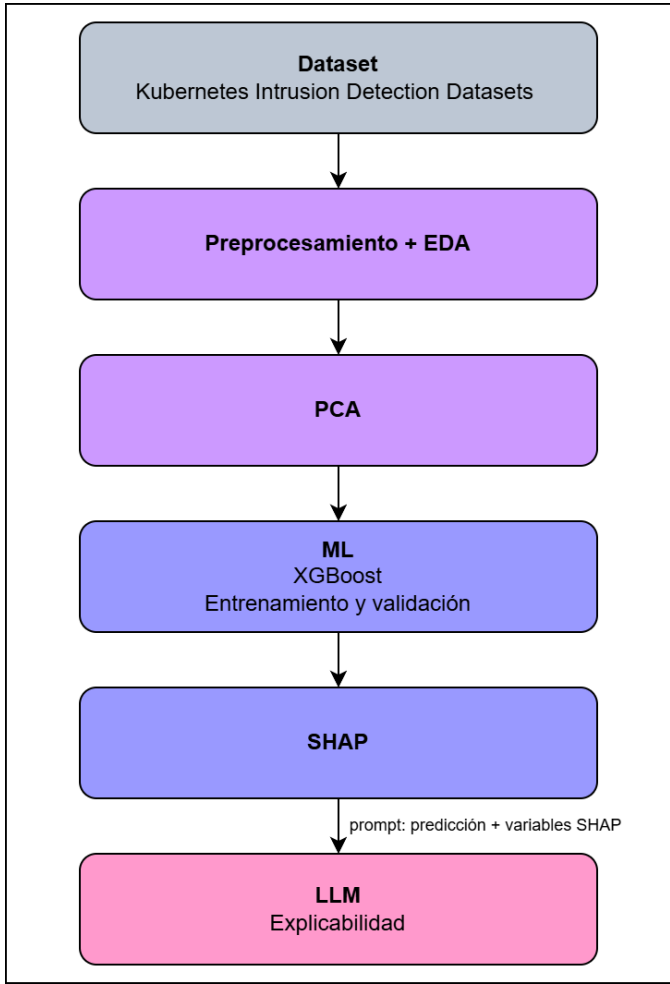


Figura 5: Arquitectura general del sistema propuesto.

3.1. Componentes principales

PCA. Inicialmente, el dataset contaba con 225 variables, de las cuales se conservaron únicamente las 177 con mayor correlación. Posteriormente, se aplicó PCA (Principal Component Analysis), reduciendo la dimensionalidad de 177 a 20 componentes principales, permitiendo disminuir la complejidad del modelo y conservar la información más representativa del sistema.

ML. El componente de Machine Learning utiliza XGBoost como modelo principal de clasificación multiclase. Este modelo se encarga de aprender patrones presentes en los datos y predecir el estado operacional del clúster Kubernetes, realizando además procesos de entrenamiento, validación y evaluación mediante métricas de desempeño como Accuracy, Precision, Recall y F1-score.

SHAP. SHAP (SHapley Additive exPlanations) se utiliza como mecanismo de explicabilidad para identificar qué componentes principales influyen más en las predicciones del modelo. Esto permite interpretar el comportamiento

de XGBoost y comprender cuáles variables tienen mayor impacto en la clasificación realizada.

LLM. Finalmente, se integra el modelo LLM Llama 3.1 8B Instant para generar explicaciones operacionales en lenguaje natural. El modelo recibe como entrada la predicción generada por XGBoost y las variables relevantes identificadas por SHAP, produciendo interpretaciones técnicas sobre el estado del clúster, posibles causas y recomendaciones básicas de monitoreo o mitigación.

4. Metodología

4.1. Configuración experimental

Cuadro 3: Hiperparámetros y configuración de entrenamiento

Parámetro	Valor
Optimizador	Gradient Boosting Tree
Learning rate	1e-3
Estimadores	100
Función de pérdida	multi:softmax
Regularización	L1 (alpha): 0, L2 (lambda): 1
Framework	xgboost, scikit-learn
Hardware	Google Colab

4.2. Estrategia de validación

Para la validación del modelo se utilizó una estrategia de división de datos en conjuntos de entrenamiento y prueba mediante train_test_split, utilizando un 80 % de los datos para entrenamiento y un 20 % para prueba. La división se realizó de forma estratificada (stratify=y) con el fin de conservar la proporción original de las clases en ambos conjuntos.

La estratificación fue especialmente importante debido al desbalance presente en la variable objetivo (label), ya que garantiza una representación adecuada de todas las clases durante el entrenamiento y la evaluación. Esto permite obtener una medición más confiable del desempeño del modelo y evitar sesgos hacia la clase mayoritaria.

4.3. Experimentos realizados

- 1. **Baseline:** Se entrenó y evaluó un modelo de Regresión Logística utilizando los datos transformados mediante PCA. Este experimento sirvió como una línea base para comparar la eficacia de otros modelos.
- 2. **Experimento 1:** Se entrenó y evaluó un modelo XGBoost utilizando los datos transformados mediante PCA. Este es el modelo principal que ya ha sido desarrollado en secciones anteriores, y sus resultados se analizarán en contraste con baseline.

5. Resultados

5.1. Métricas de evaluación

Cuadro 4: Comparación de modelos.

Modelo	Accuracy	F1 Weighted	F1 Macro
XGBoost	0.999007	0.999007	0.999218
LR	0.995035	0.995043	0.993699

5.2. Curvas de entrenamiento

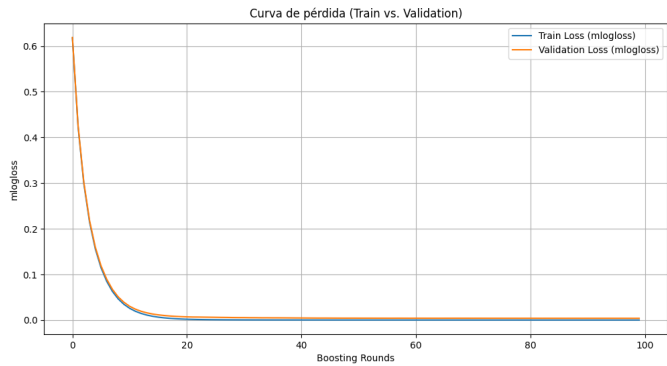


Figura 6: Curva de pérdida (train vs validation).

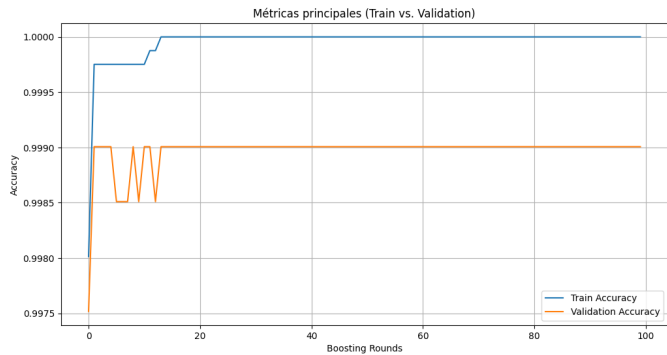


Figura 7: Métrica principal (train vs validation).

5.3. Análisis cualitativo

La matriz de confusión de XGBoost presenta un desempeño casi perfecto, al registrar solo dos errores en la clasificación de Torshammer. Esto evidencia la alta capacidad del modelo para distinguir correctamente tanto la actividad normal como los diferentes estados degradados asociados a ataques DoS.

	Pred 0 (Normal)	Pred 1 (Slowloris)	Pred 2 (Torshammer)
Real 0 (Normal)	922	0	2
Real 1 (Slowloris)	0	299	0
Real 2 (Torshammer)	0	0	791

Figura 8: Matriz de confusión de XGBoost.

De manera general, los resultados cualitativos obtenidos muestran que XGBoost ofrece un desempeño superior respecto a los demás modelos evaluados, manteniendo una clasificación más consistente entre todas las clases. Esto confirma los resultados obtenidos en las métricas cuantitativas, donde XGBoost alcanzó mayores valores de Accuracy, Precision, Recall y F1-score. Además, la combinación de PCA y SHAP permitió no solo mejorar el rendimiento del modelo, sino también incrementar su interpretabilidad al identificar los componentes principales más relevantes en cada predicción.

A continuación, se presenta el prompt utilizado para la interacción con el modelo LLM. Este prompt incluye la predicción generada por el modelo de Machine Learning, el nivel de confianza asociado y las variables más relevantes identificadas mediante SHAP. Con esta información, el LLM genera una explicación técnica en lenguaje natural orientada a la interpretación operacional del estado detectado en el clúster Kubernetes:

```
model_name = "llama-3.1-8b-instant"

prompt_text = f'''Eres un asistente especializado en
observabilidad, AIOps y diagnóstico operacional para
clústeres Kubernetes.

Tu tarea es interpretar las predicciones generadas por un
modelo de Machine Learning utilizando únicamente la
información proporcionada.

Las clases posibles son:
- 0: Normal Activity
- 1: DoS Attack 1
- 2: DoS Attack 2

La predicción realizada por el modelo es:
{prediction}

Nivel de confianza:
{confidence}

El modelo utiliza componentes principales generados
mediante PCA (Principal Component Analysis). Cada
componente principal representa una combinación de
múltiples métricas originales del sistema y del
tráfico de red.

Los componentes más relevantes según SHAP fueron:
{top_features}

Los valores asociados a cada componente principal fueron:
{features_values}

Interpreta cada componente considerando las métricas
originales asociadas a él. Analiza los patrones
generales relacionados con:
- tráfico de red,
- comportamiento de paquetes,
- uso de recursos,
- encabezados de red,
- tasas de paquetes,
- métricas temporales,
- y comportamiento operacional del clúster Kubernetes.

Genera una explicación técnica en español que incluya:
1. Interpretación del estado operacional detectado.
2. Posible impacto en el clúster Kubernetes.
3. Posibles causas del comportamiento identificado.
4. Relación entre los componentes principales relevantes y
el comportamiento detectado.
5. Recomendaciones básicas de monitoreo o mitigación.

No inventes métricas ni eventos que no estén presentes en
la información proporcionada.
```


Utiliza únicamente las variables y componentes entregados. La respuesta debe ser clara, técnica y concisa.'''

6. Discusión

6.1. Interpretación de resultados

Los resultados obtenidos muestran que el modelo XGBoost logró identificar correctamente los diferentes estados operacionales del clúster Kubernetes, alcanzando métricas de desempeño superiores al baseline basado en Regresión Logística. La combinación de reducción de dimensionalidad mediante PCA y explicabilidad con SHAP permitió mejorar tanto el rendimiento como la interpretabilidad del sistema. Además, la integración del LLM permitió generar explicaciones operacionales en lenguaje natural a partir de las predicciones del modelo y las variables más relevantes identificadas por SHAP. En general, los resultados obtenidos permiten concluir que se cumplió el objetivo planteado en la investigación, logrando una clasificación precisa y explicable de estados degradados en Kubernetes.

6.2. Limitaciones

- El dataset utilizado fue originalmente diseñado para detección de intrusiones, por lo que los estados degradados analizados están principalmente asociados a eventos de seguridad y no a fallos operacionales cotidianos de Kubernetes.
- Aunque PCA permitió reducir la dimensionalidad de 177 variables a 20 componentes principales, esta transformación dificulta parcialmente la interpretación directa de las variables originales.
- El sistema fue evaluado únicamente sobre datos estáticos del dataset, por lo que no se validó su comportamiento en escenarios de monitoreo en tiempo real.
- Las restricciones computacionales de Google Colab limitaron el uso de modelos LLM de mayor tamaño y complejidad.
- El modelo podría presentar limitaciones de generalización en entornos Kubernetes diferentes a los representados en el dataset de entrenamiento.

6.3. Trabajo futuro

Como trabajo futuro, se plantea la integración de métricas en tiempo real mediante Prometheus, permitiendo incorporar errores y anomalías operacionales comunes en Kubernetes además de los eventos asociados a seguridad presentes en el dataset actual. Adicionalmente, se propone la incorporación de arquitecturas RAG y agentes autónomos para automatizar la generación de diagnósticos, correlación de eventos y recomendaciones de mitigación dentro del clúster Kubernetes.

7. Conclusiones

El presente trabajo permitió desarrollar un sistema de observabilidad inteligente basado en Machine Learning para la detección automática de estados operacionales en Kubernetes. Los resultados obtenidos demostraron que el modelo XGBoost logró un desempeño superior al baseline de Regresión Logística, alcanzando altos valores de Accuracy, Precision, Recall y F1-score en la clasificación de actividad normal y estados degradados asociados a ataques DoS.

La aplicación de PCA permitió reducir la complejidad del dataset de 177 variables a 20 componentes principales, manteniendo la información más representativa para el proceso de clasificación. Asimismo, la utilización de SHAP mejoró la interpretabilidad del modelo al identificar los componentes con mayor impacto en las predicciones realizadas.

Finalmente, la integración del modelo LLM Llama 3.1 8B Instant permitió complementar las predicciones mediante explicaciones operacionales en lenguaje natural, facilitando la interpretación de los resultados y aportando un enfoque orientado a observabilidad y AIOps. En consecuencia, se concluye que el sistema logró identificar correctamente estados operacionales en Kubernetes con métricas de desempeño superiores al baseline planteado en la investigación.

Contribuciones del equipo

Integrante	Contribución principal
Darieth Sánchez	ML, SHAP, LLM
Valentina Benitez	Arquitectura, informe, video

Distribución de responsabilidades por integrante.

Repositorio y Demo

- **GitHub:** https://github.com/DariethSV/final_project_IA/tree/main
- **Video:** <https://canva.link/4ixujmsbf5ej0sp>
- **Notebook:** `notebook/project_AI.ipynb`

Referencias

[1] Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. <https://dl.acm.org/doi/10.1145/2939672.2939785>

[2] XGBoost Developers. (2026). *XGBoost Official Documentation*. <https://xgboost.readthedocs.io>

-
- [3] Jolliffe, I. T. (2002). *Principal Component Analysis*. Springer. <https://link.springer.com/book/10.1007/b98835>
 - [4] Molnar, C. (2020). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. <https://christophm.github.io/interpretable-ml-book/>
 - [5] Lundberg, S. M., & Lee, S. I. (2017). *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems (NeurIPS 2017). <https://arxiv.org/abs/1705.07874>
 - [6] SHAP Developers. (2026). *SHAP Official Documentation*. <https://shap.readthedocs.io>
 - [7] Stallings, W. *Cryptography and Network Security: Principles and Practice*. Pearson Education.
 - [8] Kaggle. (2026). *Kubernetes Intrusion Detection Datasets*. Kaggle. <https://www.kaggle.com/datasets>